

Universidad Tecnológica Nacional – Facultad Regional Rosario
Ingeniería en Sistemas de Información
Cátedra de Redes de Información

CAPA DE RED
PROTOCOLO UDP

Ing. Hernán Vitri
Ing. Germán Baró
Ing. Esteban Travaglino

CAPA 4 - PROTOCOLO DE TRANSPORTE UDP

- ÍNDICE:

1- Encabezamiento UDP.

2- Pseudoencabezamiento IP.

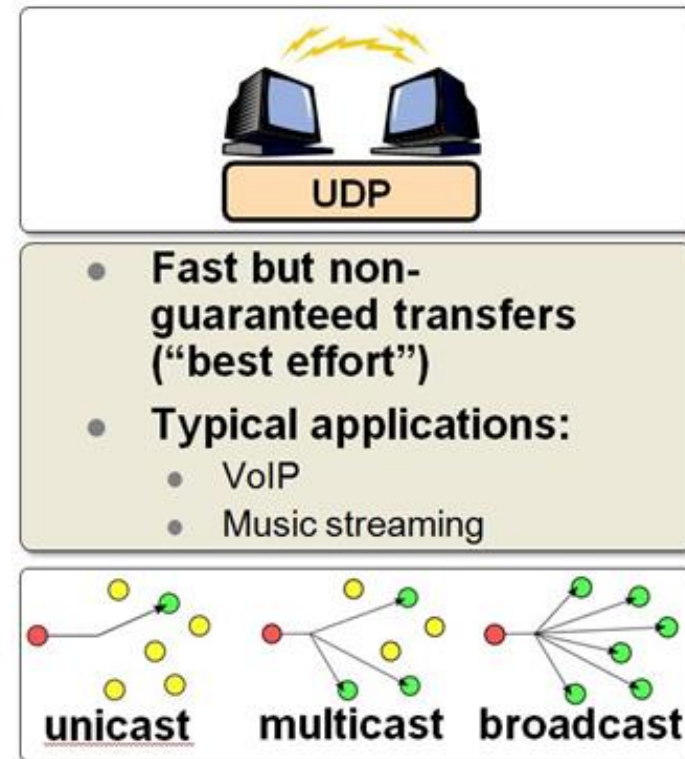
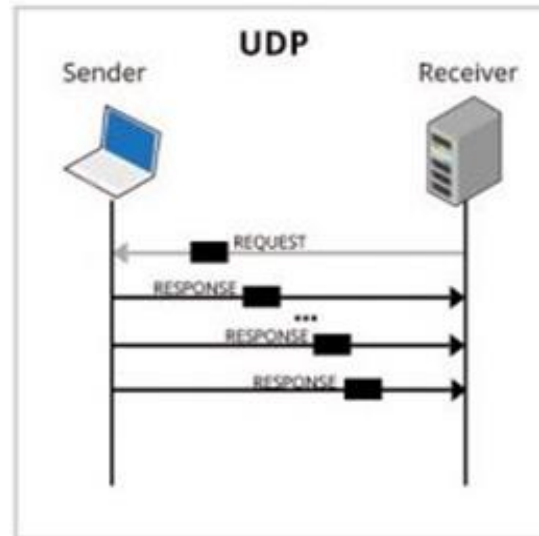
3- Procesos Remotos.

4- Transporte de datos en tiempo real.

5- RTP, "Real-Time Protocol".

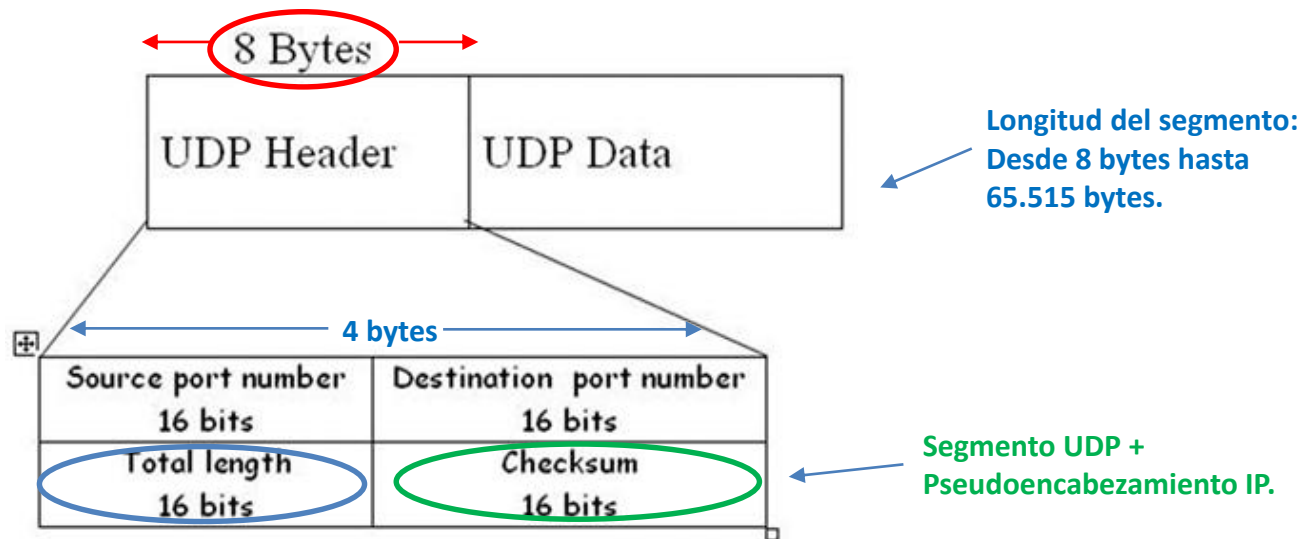
6 - RCTP, "Real-time Transport Protocol".

7- Bibliografía.



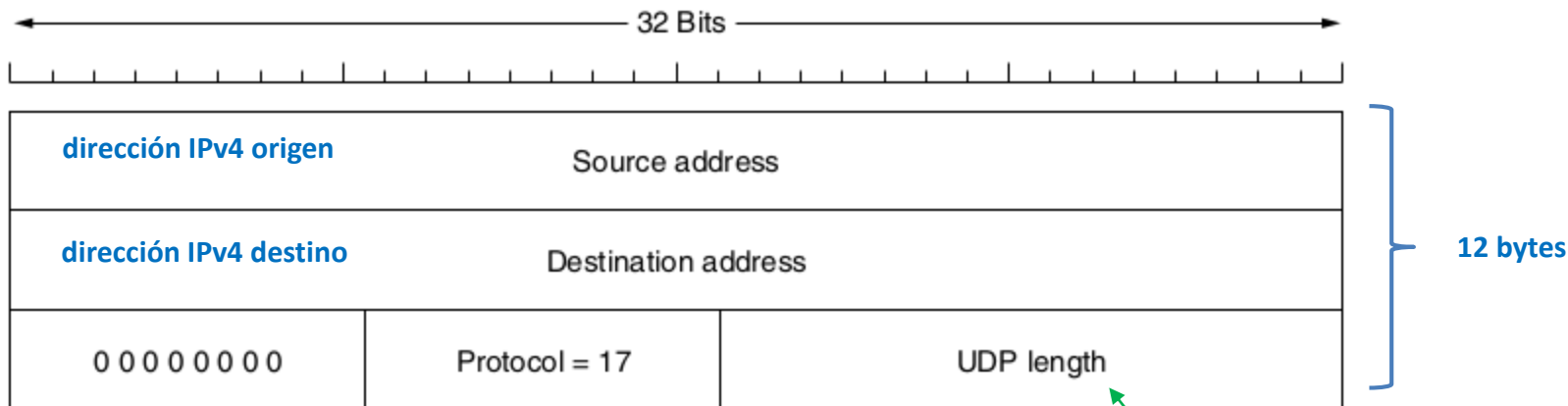
1 - ENCABEZADO UDP (User Datagram Protocol).

- **Protocolo UDP:** Brinda servicios sin conexión. Simplemente envía segmentos de puerto a puerto, dejando que las aplicaciones en capas superiores se encarguen de gestionar errores y controlar el flujo. Está definido en la RFC 768.
- **Encabezamiento UDP:** En la figura se observa el encabezado del segmento UDP, que consta de 8 bytes (= 64 bits). 2 bytes son usados para el puerto origen, y 2 bytes para el puerto destino. La longitud mínima del segmento UDP es 8 bytes, (sólo el encabezado), mientras que la longitud máxima es 65.515 bytes, algo menor que 64 KB ($= 2^{16} = 65.536$), para que el segmento UDP más largo quepa en el tamaño máximo de un paquete IP.
- **Checksum opcional:** Si detecta errores, no corrige pero avisa a las capas superiores. Este “checksum” revisa el segmento UDP completo más un pseudoencabezamiento IP, que ahora veremos. El algoritmo suma las palabras de 16 bits de longitud en complemento a uno y toma el complemento a uno de esta suma. Como consecuencia, cuando el receptor realiza este “checksum” incluyendo el campo “checksum”, el resultado debe ser cero. Si el “checksum” no se calcula, lleva todos ceros, que en complemento a uno se transforma en unos. (No es conveniente desactivar el checksum, a menos que no sean importantes los datos transportados, por ejemplo, una comunicación de voz sobre IP, VoIP).



2 – PSEUDOENCABEZADO IP.

- **Violación de la estructura de capas:** La inclusión del pseudoencabezamiento IP ayuda a detectar paquetes de capa 3 con errores, aunque constituye una violación en la estructura de capas del modelo OSI, porque la capa 4 controla parte de la capa 3.
- El pseudoencabezamiento IPv4 se observa en la figura. Está formado por las direcciones IPv4 de 32 bits, luego sigue un byte de ceros, el número de protocolo UDP (= 17) y la longitud del segmento UDP.
- UDP puede detectar errores con su checksum, pero deja que las capas superiores decidan qué hacer el respecto. Tampoco realiza control de flujo ni retransmisiones de segmentos. UDP únicamente gestiona puertos distintos para separar comunicaciones distintas, y deja que se encargue la capa de aplicación. UDP es simple, no necesita un proceso de establecimiento de conexión, ni de desconexión, como TCP, simplemente se envía un mensaje en cada dirección.

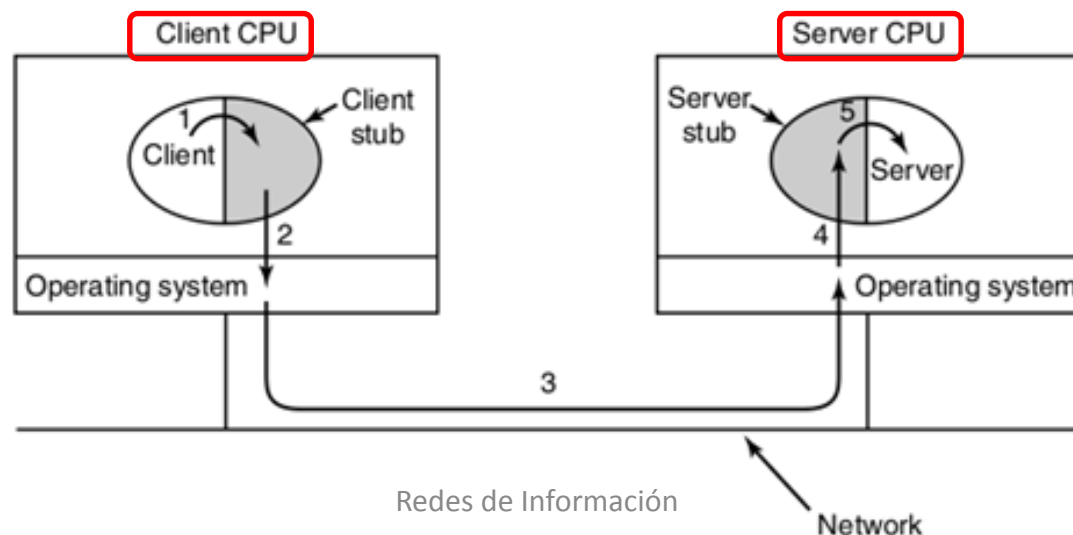


The IPv4 pseudoheader included in the UDP checksum.

Longitud del Campo de datos
del paquete IP =
= ILH – IP Total Length.

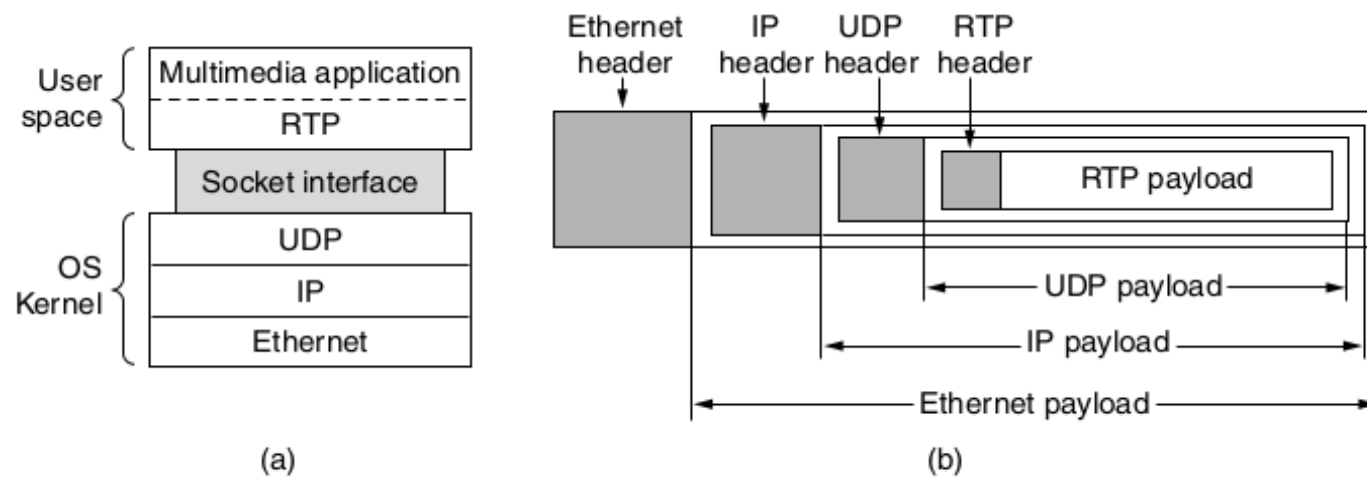
3 – PROCESOS REMOTOS.

- **RPC (Remote Procedure Call):** Es muy común en capa 4 que un host solicite ejecutar un proceso en otro host remoto. Para las capas superiores, parece un proceso local en el mismo host. Esto se conoce como RPC. El host 1 suele llamarse “cliente” y el proceso o host 2, “servidor”. Sucede por ejemplo al solicitar una canción en Spotify.
- En la figura se observa un ejemplo que se detalla a continuación:
 1. El cliente ejecuta localmente una primera parte del proceso, llamado “client stub”. (stub = colilla, cabo).
 2. Este proceso solicita el envío del segmento al servidor, “marshaling”.
 3. El sistema operativo del cliente envía el segmento.
 4. El sistema operativo del servidor pasa el mensaje al proceso correspondiente en el servidor, “server stub”.
 5. El “server stub” solicita al servidor propiamente dicho, los parámetros pedidos.
- Luego se envía la respuesta hacia el cliente con pasos similares en dirección opuesta. Este mecanismo de entrada/salida es distinto al mecanismo ejecutado mediante “sockets”. UDP es útil para transportar todos estos procesos, con algunas restricciones.



4 – TRANSPORTE DE DATOS EN TIEMPO REAL.

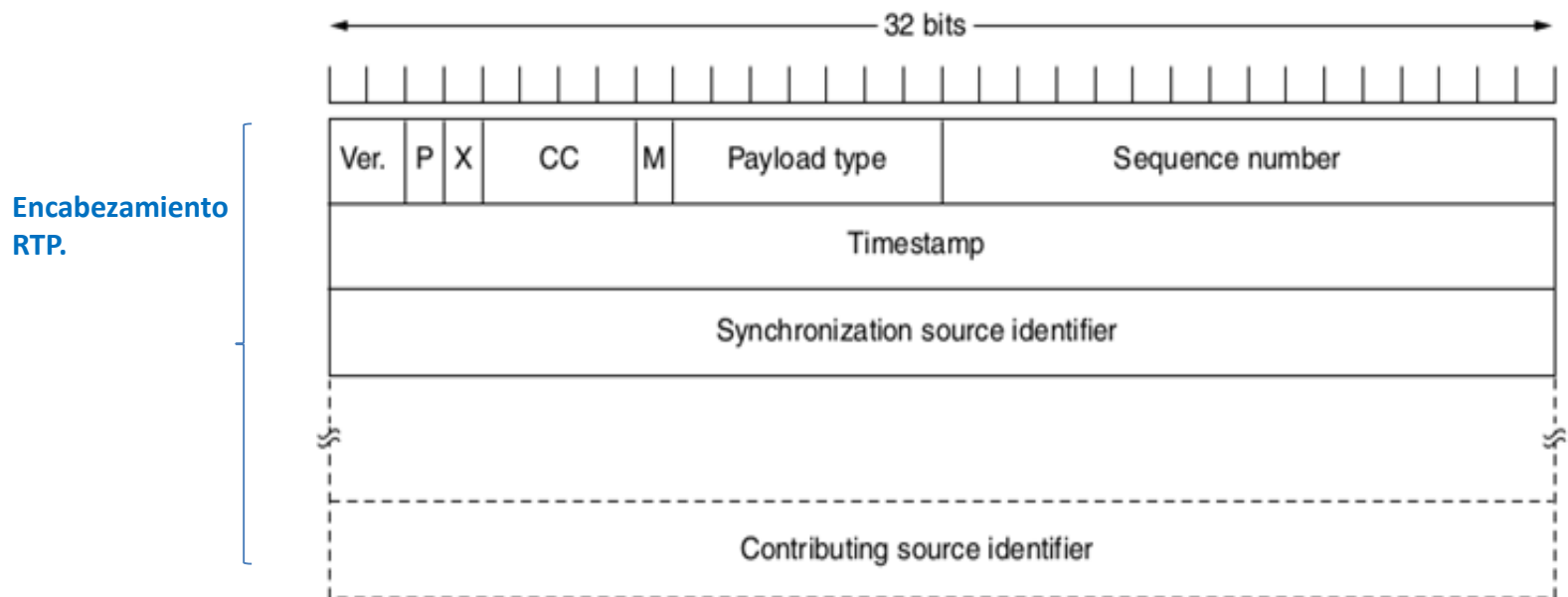
- **Streaming:** En aplicaciones multimedias en tiempo real se emplea UDP. Por ejemplo, para escuchar radio o música por internet, en telefonía sobre IP, videos y teleconferencias,etc.
- Además de UDP, se utiliza el protocolo RTP (Real-Time Protocol), descrito en la RFC 3550. En la figura se observan las ubicaciones de los protocolos en el modelo OSI.
- **RTP:** La función básica de RTP es juntar varios paquetes de datos en tiempo real y encapsularlos en segmentos UDP. Si se pierde un paquete RTP, es problema de la aplicación qué hacer al respecto. Como son aplicaciones en tiempo real, una retransmisión del paquete perdido no tiene sentido, porque llegaría tarde para ser útil, y por eso RTP no tiene mecanismos de retransmisión ni envía notificación por cada trama recibida, (tampoco UDP).



(a) The position of RTP in the protocol stack. (b) Packet nesting.

5. RTP (Real-Time Protocol).

- **Codificación:** RTP define varios perfiles de “streaming”, (o cadenas de datos en tiempo real). Cada perfil está pensado para un servicio diferente y permite varias maneras de codificar los datos.
- ✓ Por ejemplo, una canción en Spotify puede ser codificada como MP3, o codificación GSM, o en 8-bit PCM, etc. Pero hay sitios web que ofrecen mejor calidad de sonido empleando codificaciones de mejor calidad, por ejemplo: Tidal.com.
- **Sincronismo:** También es importante el instante de tiempo en que se codifica una muestra, ya que el receptor debe respetar este instante al decodificar los datos, independientemente del retardo que tuvo en la red, para que el mensaje sea reproducido tal como fue grabado.
- ✓ Por ejemplo, una película en Netflix tiene varias cadenas de datos relacionados: video, voz en español, voz en inglés, etc. Todas estas cadenas de datos deben estar perfectamente sincronizadas en el receptor.



5. RTP (continuación).

Encabezamiento: Se observa en la figura el encabezamiento RTP.

“**Ver**” = “Version” = 2.

“**P**” = “Padding” = relleno.

“**X**” = Extension header is present.

“**CC**” = cantidad de fuentes de streaming.

“**M**” = Bit usado por la aplicación en capa 7, por ejemplo, para indicar un comienzo de muestra de audio.

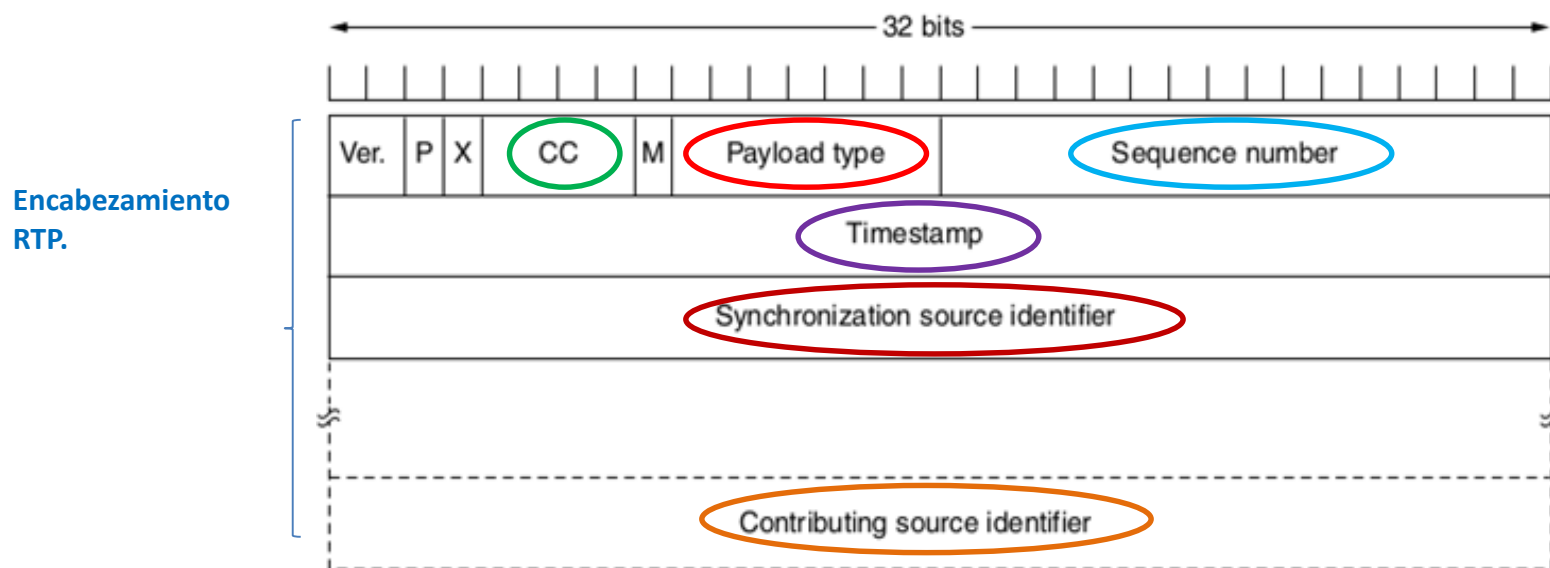
“**Payload Type**” = Algoritmo de codificación de los datos, MP3, MP4, etc.

“**Sequence number**” = Número de secuencia para identificar el paquete.

“**Timestamp**” = Instante de inicio de la muestra. Importante para mantener el sincronismo y reducir el jitter.

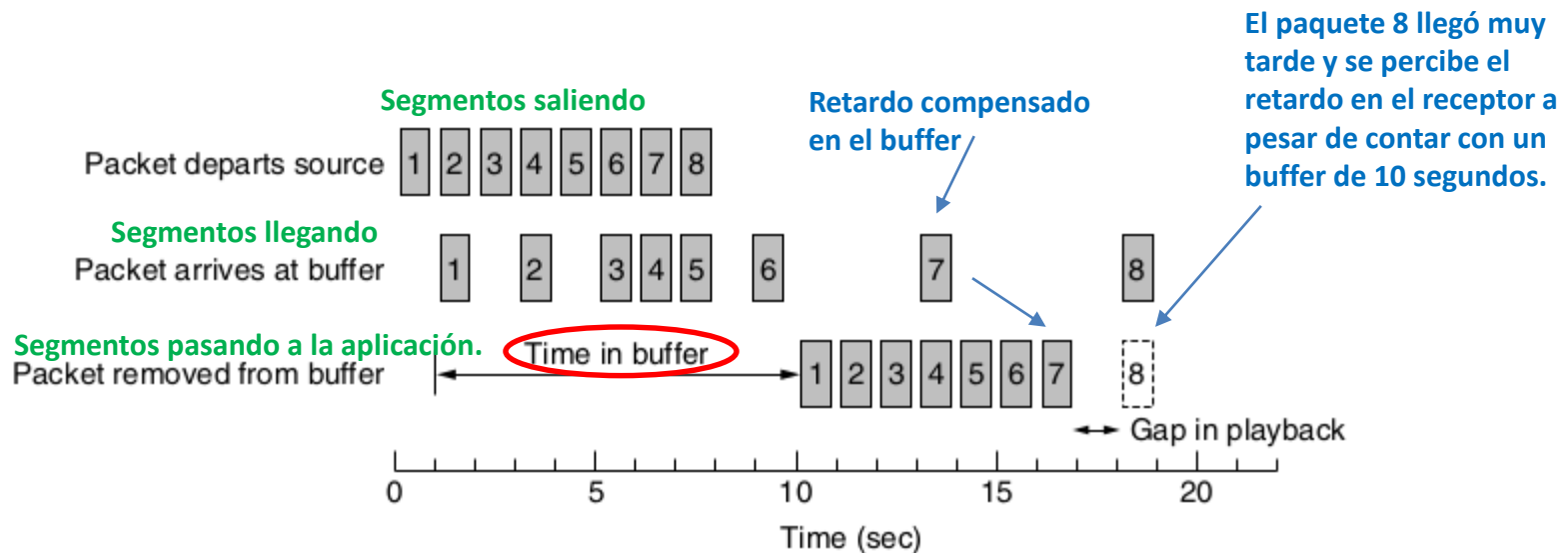
“**Synchronization source identifier**” = Indica a qué fuente de streaming pertenecen los datos transportados.

“**Contributing source identifier**” = Indica los distintas fuentes de streamings que integran una mezcla, si es que existe.



6 - RTCP (Real-time Transport Control Protocol).

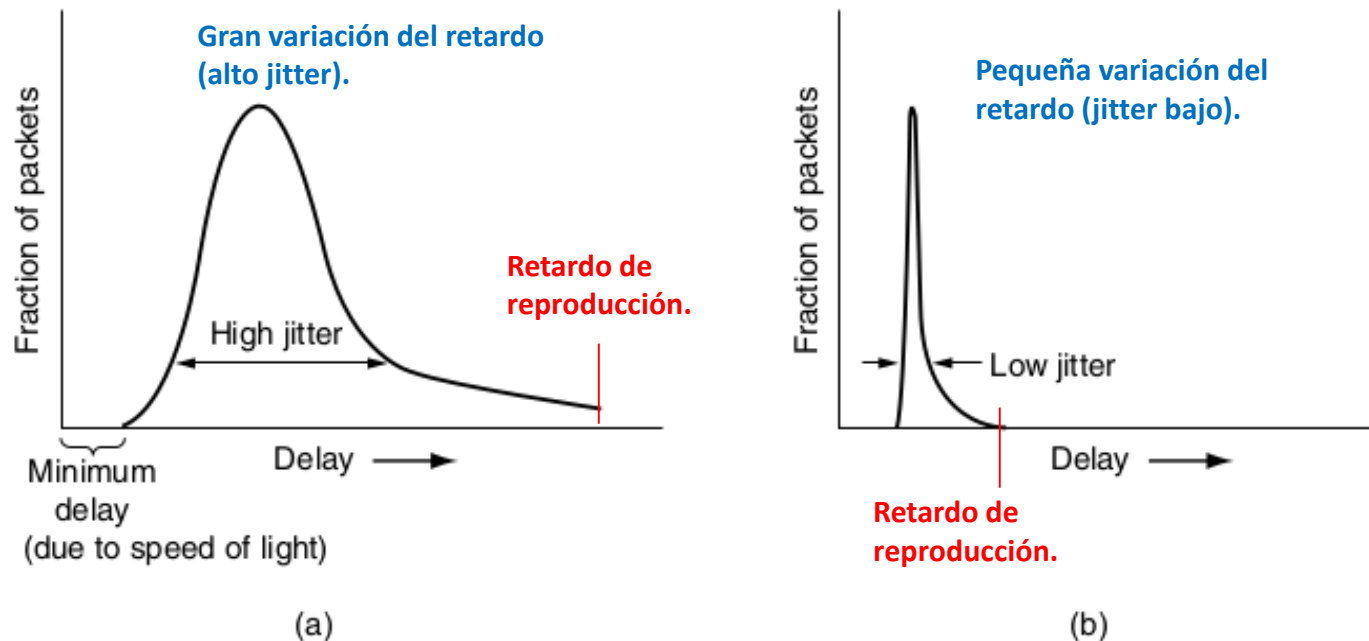
- **Sincronismo y feedback:** Este protocolo de control se encarga de la sincronización y el feedback de la interfaz de usuario en las transmisiones en RTP. La información enviada por RTCP puede ser utilizada para bajar la velocidad (y calidad) cuando hay problemas en la red y viceversa.
- **Buffers:** En la figura observamos la importancia del buffer de almacenamiento en el receptor.
 - ✓ En streamings de audio y video en una dirección, es común emplear un buffer de 10 segundos.
 - ✓ En videoconferencias en tiempo real, en cambio, el buffer debe ser mucho menor a 10 segundos para permitir interacciones desde ambos lados de la conversación.



Smoothing the output stream by buffering packets.
Importancia del buffer para mantener el sincronismo.

6 - RTCP (Real-time Transport Control Protocol)

- Jitter:** En estas aplicaciones se estudia el jitter, que es la variación del retardo entre varios segmentos, para definir el retardo adecuado. Deberíamos tomar un punto de playback, o buffer, que contenga al 99% de las muestras de datos, para lograr una adecuada reproducción. En las figuras vemos dos casos de jitter diferentes, que se suman al tiempo de propagación.



(a) High jitter. (b) Low jitter.

7- BIBLIOGRAFÍA.

- Tanenbaum, Andrew y Wetherall, David. Computer Networks. 5th edition. Pearson Education Inc.